

DEPENDENCY SECURITY BEFORE EXECUTION

WARDEN

Every dependency change becomes
a verified transaction.

Plan the complete graph. Approve only the authority required. Install
with scripts suppressed. Prove what landed.



```
$ npm install esbuild
```

THE INDUSTRY BLAST RADIUS

A package can be poisoned for hours and still reach the internet at ecosystem scale.

NEW MALICIOUS PACKAGES IDENTIFIED IN 2025

454,648

More than one new malicious package surfaced every 70 seconds.



REACH

2B+ downloads each week

chalk, debug, and related packages before the September 2025 compromise



PROPAGATION

796 packages, 1,092 versions

carried the self-propagating Shai-Hulud 2.0 campaign



CONSEQUENCE

30% of breaches

involved a third party in Verizon's 2025 breach dataset

Short exposure does not mean small impact.

Ecosystem reach turns one compromised release into an industry response.

One missing letter redirects trust to a different package.

PACKAGE UNDER REVIEW

expres, different package

Refresh live data

CAPTURED NPM DATA

npmjs.com/package/expresopen npm page		
PACKAGE NAME expres missing the final "s" latest 0.0.5	DOWNLOADS LAST WEEK 7,348 18 Jul to 24 Jul 2026	WHAT IT IS Add express compatible methods to your response object SOURCE github.com/cpsubrian/node-expres IDENTITY RESULT Different package Real package, but not the official Express web framework.

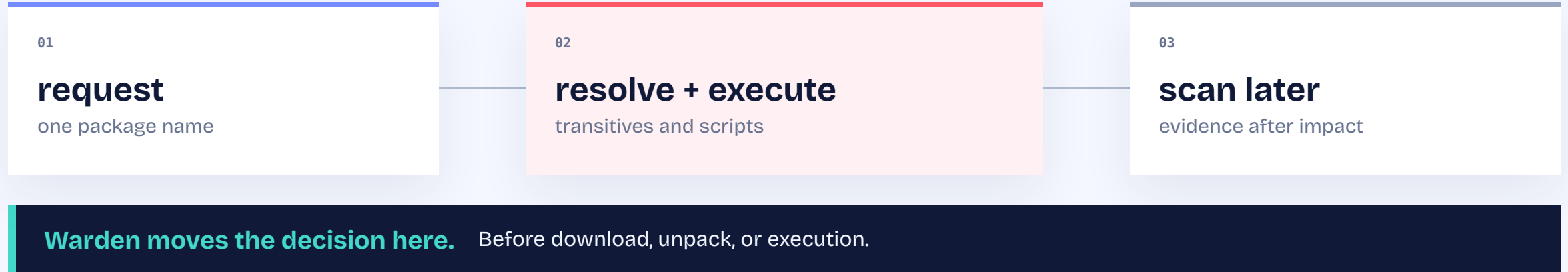
Both names resolve.

Warden checks name similarity, provenance, release history, scripts, and artifact behavior before installation.

THE PROBLEM

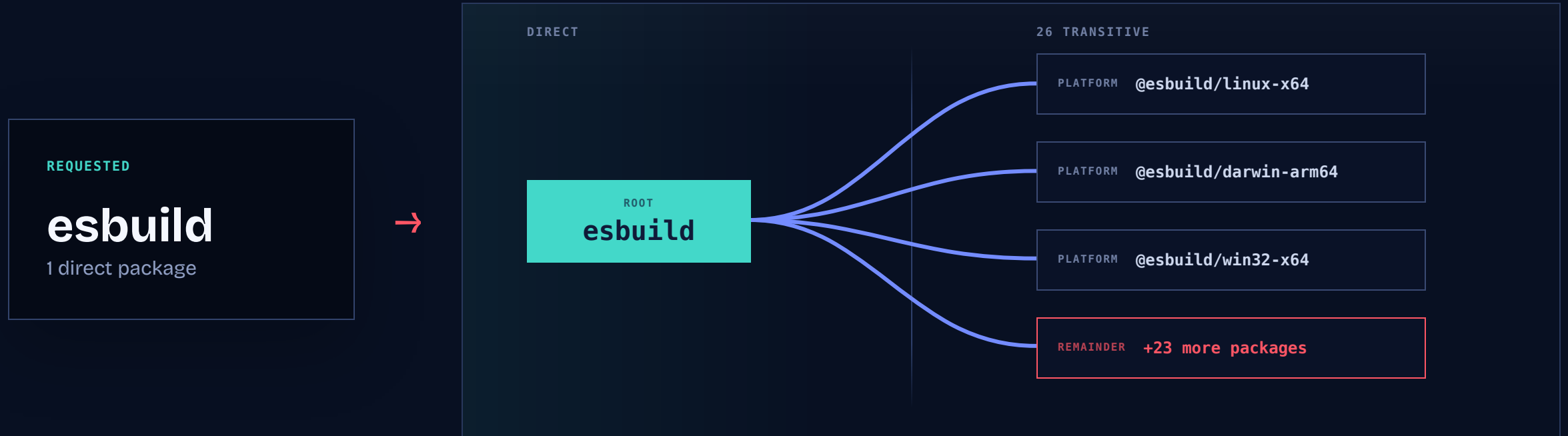
The dangerous moment happens before a scanner can report it.

A lifecycle script runs with your permissions. By the time a post-install scan starts, credentials or source may already be gone.



THE HIDDEN CHANGE

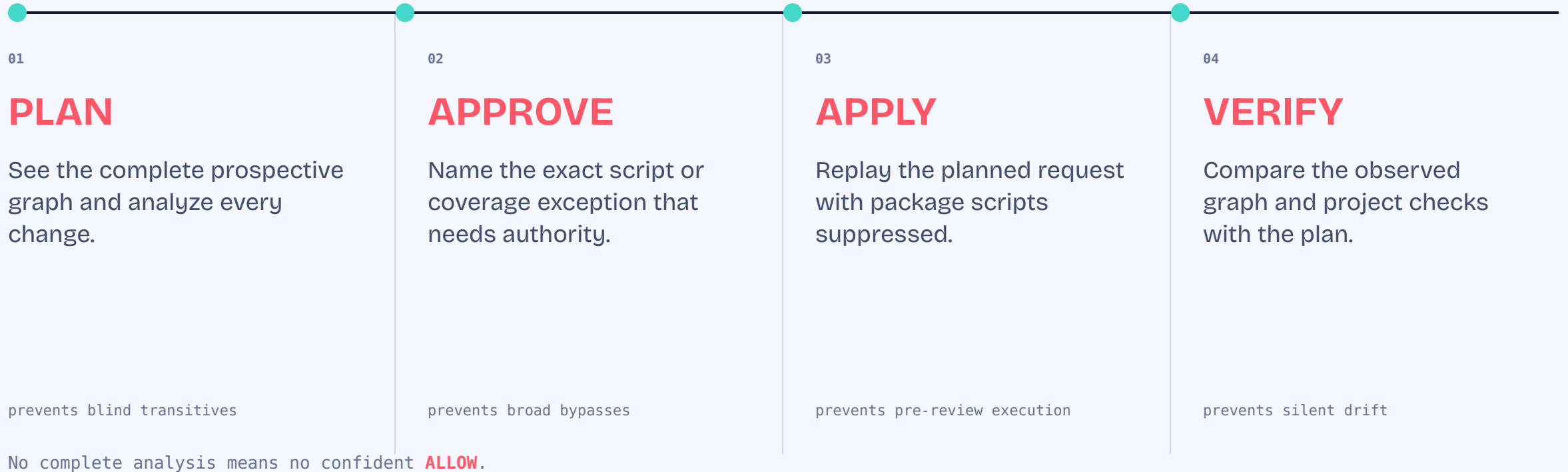
You type one package. The manager installs a graph.



**27 packages
change**

Checking only the name you typed leaves 26 transitive packages unexamined.

Four transaction stages close four different failure modes.



A package name does not reveal the graph, scripts, or authority it will add.

● ● ● prospective graph

▶▶ SKIP ● LIVE

```
$ warden plan -- npm install esbuild
✓ resolving the prospective dependency graph · 27 6.1s
✓ vetting changed packages · 27/27 19.6s
Warden plan: npm install esbuild
+ 26 transitive packages · 27 of 27 analyzed
NEEDS_APPROVAL esbuild@0.28.1 has a postinstall script
approval binds version + integrity + hook + script body
script policy: suppressed
```

REQUEST SHOWS

1 name

the direct package only

WARDEN PLAN REVEALS

27 changes

all 27 artifacts analyzed

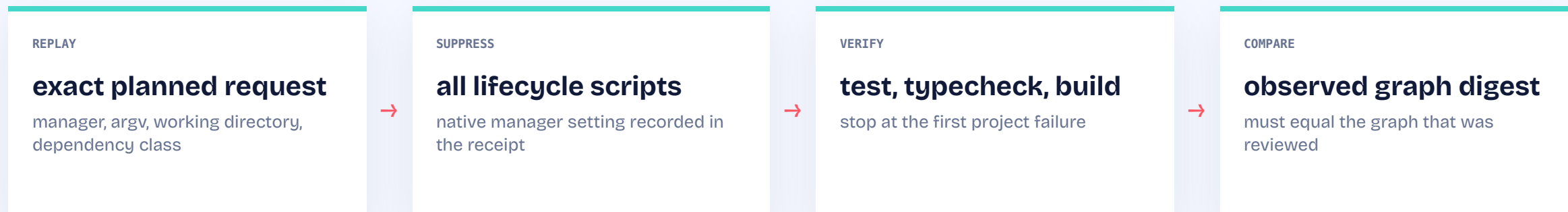
SAFE NEXT STEP

1 approval

bound to the exact postinstall hook

`warden plan` turns an install request into a reviewable transaction. The script still does not run.

A safe plan is worthless if installation resolves something different.



TRANSACTION RECEIPT

result: applied

graph reviewed = graph observed
script policy = suppressed
artifact coverage = complete

WHEN ANY CHECK FAILS

Restore the manifest and every lockfile.

Warden does not claim to restore node_modules or side effects from verification.

A local guard can be bypassed. The pull request still needs proof.

GUIDANCE	Teach the safe loop	Instructions and agent setup make the intended path obvious.	can be ignored
INTERCEPTION	Mediate package-manager commands	Shims cover npm, pnpm, Yarn, Bun, npx, bunx, exec, dlx, ci, rebuilds, and global installs.	absolute paths and containers can bypass it
VERIFICATION	Refuse the pull request without a valid receipt	<code>warden ci --require-transaction-receipt</code>	does not trust the developer machine



A trusted package name says nothing about this release's code or source.

PUBLISHED PACKAGE

**resolve → integrity → release diff → AST
scan → name intel → deterministic score**

Finds known malware, name attacks, new execution, provenance changes, credential access, egress, destructive calls, and obfuscation combined with capability.

`warden check` is deterministic. Verdicts are cached by tarball integrity and analyzer version, so repeat checks stay fast without reusing stale analysis.

LOCKFILE

Where will bytes come from?

Off-registry hosts, plaintext transport, weak integrity, git and file sources.

SCRIPTS

What code runs during install?

Lifecycle hooks across the installed graph, including pipe-to-shell and credential access.

CONFIG

Where will credentials be sent?

Lookalike registries, plaintext tokens, and disabled TLS without echoing secret values.

One safety policy becomes four opportunities for settings to diverge.

REPOSITORY INTENT

approved scripts

minimum release age

block exotic sources

reverify lockfiles

block downgrades

→

npm	ignore-scripts, min-release-age, source controls
pnpm	strict builds, minimum age, exotic-subdependency controls
Yarn	script control, hardened mode, age gate
Bun	install script suppression

HONEST GAP REPORTING

If the manager cannot express a rule, Warden says so and enforces it in the plan and receipt gate.

The invoked manager, packageManager field, lockfile, project config, and PATH decide which tool Warden preserves.

A vulnerability fix is not safe until the candidate release passes the gate.

01 Find advisories

Audit direct dependencies through OSV.

02 Build repair plans

Compare minimal and latest candidates.

03 Gate every fix

Reject the official fix if its release is risky.

04 Verify in isolation

Install without scripts, run project checks, then apply.

● ● ● repair plan

▶▶ SKIP

● LIVE

```
$ warden doctor
auditing 3 direct dependencies against OSV advisories
critical acme-http@1.0.0 fixed in 1.0.1
BLOCK acme-http@1.0.1 new script + exfiltration + provenance downgrade
UNFIXABLE the official fix fails the supply-chain gate
verified acme-json@2.1.4 install ok · test ok · applied
1 of 2 issues fixed · exit 10
```

Code can compile and still fail the request.

prompt versus diff

SKIP LIVE

```
$ warden intent check
prompt "add rate limiting, keep retries, log every limited request"
3 claims extracted · 2 files changed
✓ delivered    add rate limiting
✓ preserved    retry logic untouched
x dropped      log every limited request
! scope creep  pagination.ts was never requested
! hallucinated axios.instance.throttle is not exported
! unpublished  import "fetch-retry-helper-pro" has never existed on npm
the last two need no model · llm calls: 2
BLOCK diff does not match the prompt · exit 20
```

DROPPED
A requested behavior never appeared.

SCOPE CREEP
Unrequested files or behavior changed.

HALLUCINATED API
Added code calls an export that does not exist.

UNPUBLISHED IMPORT
An added import names a package the registry has never had.

The last two are deterministic: no model, no tokens, and they still run when no model is configured at all. When `.warden/prompt.txt` exists, CI runs the same check for changed JavaScript and TypeScript.

Automation needs a decision contract, not terminal prose.



Stable contract	JSON output, published schemas, typed errors, and exit codes 0, 10, 20, 30.	Safe data boundary	Registry-authored text is sanitized and quarantined as untrusted input.
Capability-aware setup	Adapter setup reports which guidance, interception, post-change, and tool layers are actually available.	Restricted tools	The generated tool manifest exposes read-only decisions and excludes project-changing commands.

`warden handoff` writes the failing finding, evidence, concrete fix, and `warden ci --reporter agent` finish line

WHEN SOMETHING FAILS

Start with the operational problem. The command follows.

FAILURE	WARDEN RESPONSE	PROOF PRODUCED
RISKY RELEASE The team needs evidence and a safer candidate.	check explain history compare baseline scripts	release verdict evidence trail, candidate ranking, and exact remediation
BROKEN GUARDRAIL Interception or repository setup may be incomplete.	coverage integrations doctor detect init config log	coverage and repair missing layers, concrete fixes, and recorded verdict history
MISSING SYSTEM PROOF CI and tools need stable, machine-readable evidence.	install ci schema completions benchmark uninstall	repeatable operations manager-preserving execution, report contracts, and lifecycle control

MEASURED AND BOUNDED

The evidence is reproducible. The limits are explicit.

12/12

curated attack shapes
stopped

0/9

benign shapes stopped

METHOD

Every case runs through the real resolver and transaction decision with complete graph coverage.

Regression corpus, not field accuracy.

INTENT, MEASURED PER RULE · warden intent bench

100%

unpublished import, precision

100%

hallucinated API, precision

60%

claim matching, precision

18.2%

false positives, against a stated 5%
budget

Published because it fails the budget. The deterministic rules block; claim matching is advisory until it earns it.

Not a sandbox

Absolute paths and containers can bypass local shims.

Scripts stay suppressed

Packages that require install hooks may fail verification.

Rollback is scoped

Manifest and lockfiles return. node_modules does not.

Receipts are unsigned

Evidence, not independent attestation.

Technical alpha: a working control plane with honest boundaries.

THE OUTCOME

Keep the workflow. Add a chain of custody.



ONE DECISION CONTRACT

ALLOW • WARN • NEEDS_APPROVAL • BLOCK

local, package managers, automation, and CI

EVIDENCE

Sources

01 Sonatype software supply-chain report

03 Datadog Shai-Hulud 2.0 analysis

05 USENIX package hallucination study

07 Warden benchmark corpus and method

09 Command interception coverage

02 GitHub package compromise reach

04 Verizon 2025 breach report

06 OSV advisory database

08 Transaction model and limitations

10 npm live package comparison

Product claims correspond to the shipped command registry and repository documentation on this revision.